

API Attack-Path & Authorization Tester

Project Documentary & Interview Deep-Dive

How the system was designed, implemented, tested, and demonstrated

Project type	Deterministic, OpenAPI-driven API authorization security tester
Core language	Python
Primary focus	BOLA/IDOR, BFLA, privilege escalation, parameter tampering, business-logic abuse
Execution	CLI-first, scriptable, Docker-ready, with an intentionally vulnerable Flask lab

Important honesty note

This document is written as an interview-ready first-person narrative. Use first-person claims only for the portions you personally implemented or can confidently defend. The repository can show what the system does; it cannot prove which lines you personally authored.

Repository reviewed: <https://github.com/d3zathon/claude-AATi-test>

Prepared: 17 August 2026

Contents

Section	What it covers
1. The project in one sentence	The elevator pitch and boundaries of the tool.
2. The problem I was solving	Why generic scanning is not enough for API authorization.
3. What I built	Architecture, repository layout, and technology choices.
4. How I built the system	The implementation story from OpenAPI parsing to reporting.
5. The five security checks	BOLA, BFLA, privilege escalation, parameter tampering, business logic.
6. Verification and evidence	Why candidates are not immediately reported as vulnerabilities.
7. Attack-path correlation	How confirmed findings become an impact narrative.
8. The vulnerable lab and Docker	How I made the project reproducible and demonstrable.
9. Testing strategy	How false positives and false negatives are tested.
10. A complete demo walk-through	What happens when I run the scanner from the CLI.
11. Design decisions and trade-offs	Why the architecture looks the way it does.
12. Limitations and roadmap	What the project does not do and what comes next.
13. Interview presentation script	A polished 60-90 second explanation.
14. Questions I must be able to answer	Likely technical interviewer questions and strong answers.
15. Glossary	Terms worth knowing before the interview.

1. The project in one sentence

My elevator pitch

“I built a deterministic API security testing tool that starts from an OpenAPI specification and multiple role-based credentials, tests authorization boundaries for common API flaws, verifies suspicious results with real HTTP evidence, and correlates confirmed findings into attack paths.”

The key word is authorization. This is not a generic scanner for every web vulnerability. The project is deliberately narrow: it tries to answer whether the API correctly enforces who can access an object, who can call a function, who can change privileged properties, and whether sensitive business workflows can be abused.

Question	What the project does
What is the attack surface?	OpenAPI 3.x paths, operations, parameters, request bodies, and declared access-control hints.
Who are the attackers?	Configured test personas with auth headers/tokens, ownership, identity markers, and privilege ranks.
What is tested?	BOLA/IDOR, BFLA, privilege escalation, parameter tampering, and workflow abuse.
How is a finding trusted?	It is re-issued, its response content is inspected, and special capability checks are performed where needed.
How is impact communicated?	Confirmed findings can be correlated into grounded attack paths and rendered as Markdown, HTML, and JSON.

Boundary of the project

I should not describe this as an all-purpose DAST platform, an SQL injection/XSS scanner, or an autonomous AI pentesting agent. Those are explicitly outside the project scope.

2. The problem I was solving

Why authorization deserves its own testing workflow

The starting problem is simple: many API security failures are not caused by a broken cryptographic primitive or a missing header. They are caused by the server answering an authenticated request without correctly checking whether that identity is allowed to access the specific object or function.

```
GET /api/orders/101
Authorization: Bearer <alice-token>

Question: does Alice own order 101?
```

A scanner that treats “HTTP 200” as proof of success can be badly misleading. Applications can return an HTTP 200 page that contains an access-denied message, an empty response, or a generic error object. The project therefore separates “this looks suspicious” from “this is confirmed.”

Traditional weak signal	Project response
HTTP 200	Treat 200 as a candidate, not proof.
Random object-ID fuzzing	Use configured resource ownership and role pairs.
PATCH returned 200	Re-probe a restricted endpoint to see whether privilege actually changed.
Workflow cannot be inferred	Make business workflows explicit in configuration.
A list of independent CVEs/findings	Correlate confirmed findings into realistic attack paths.

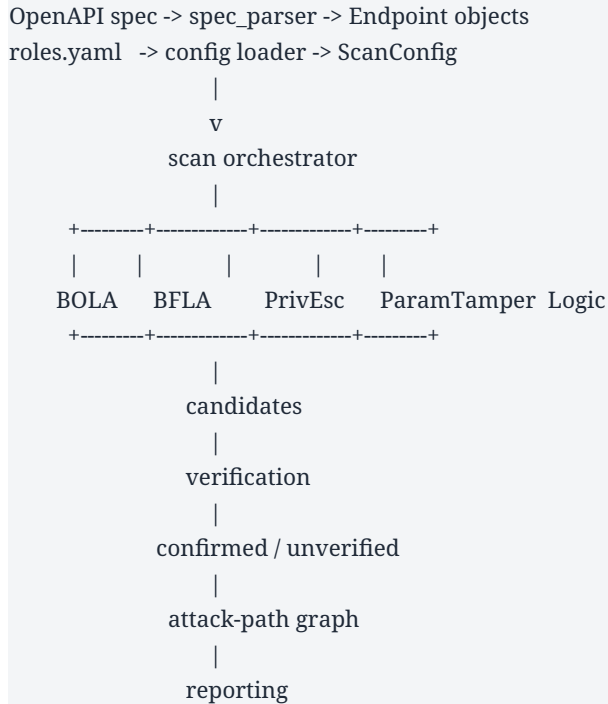
The design principle

The tool is intentionally deterministic and auditable. A reviewer can inspect the exact request/response evidence that caused a finding to become confirmed.

3. What I built

Architecture and repository structure

The project is a Python package exposed as a CLI. The code is organized into a pipeline rather than a single large scanner function.



Repository area	Responsibility
src/apiattack/spec_parser.py	Converts OpenAPI data into normalized endpoint objects.
src/apiattack/config/	Loads role, ownership, privilege, endpoint-requirement, and workflow configuration.
src/apiattack/engine/	HTTP client plus scan orchestration.
src/apiattack/checks/	Separate test modules for the five vulnerability classes.
src/apiattack/verification/	Re-tests candidates and decides confirmed vs unverified.
src/apiattack/attack_path/	Builds grounded chains from confirmed findings.
src/apiattack/reporting/	Produces Markdown, HTML, and JSON reports.
lab/	Intentionally vulnerable Flask target with seeded flaws and safe controls.
examples/	OpenAPI spec, role configuration, and sample report.
tests/	Unit and integration tests, including tests against the lab.

The repository README and architecture document describe this pipeline directly. The architecture file names the key functions: `parse_spec()`, `load_scan_config()`, `run_scan()`, `verify_all()`, `build_attack_paths()`, and report writers.

4. How I built the system

The implementation story from input to report

4.1 Start with a structured API description

I chose OpenAPI as the entry point because the tool needs a reliable inventory of paths, HTTP methods, parameters, and request bodies. Instead of crawling blindly, I can ask the specification what the API says exists.

```
GET /users/{user_id}
POST /users
PATCH /users/{user_id}
DELETE /users/{user_id}
```

The parser normalizes these operations into endpoint objects. It can also use x-required-roles declarations as an input to authorization testing.

4.2 Define test personas and the authorization model

Authorization cannot be tested with one credential. I defined role-based test identities and recorded the information the scanner needs to reason about them.

```
roles:
- name: user
  privilege_rank: 1
  auth_header: ...
  owned_resources: ...
  identity_markers: ...

- name: admin
  privilege_rank: 9
  auth_header: ...
```

- owned_resources tell the scanner which objects a persona legitimately owns;
- privilege_rank helps avoid using a legitimately powerful identity as a BOLA attacker;
- identity_markers help verify that the response actually contains another principal's data;
- endpoint_role_requirements define who should be allowed to call restricted operations;
- workflows define business sequences that cannot be inferred safely from OpenAPI alone.

4.3 Build a reusable HTTP layer

All API calls go through a single HTTP client abstraction. This centralizes timeouts, authentication headers, body handling, timing, and evidence capture instead of duplicating request code inside every check.

4.4 Add focused check modules

Each check receives normalized endpoints and configuration, runs its test strategy, and emits a Finding when the response is suspicious enough to investigate. Importantly, the check itself does not decide that the finding is confirmed.

5. The five security checks

What I test and how the reasoning works

5.1 BOLA / IDOR

BOLA means Broken Object Level Authorization. IDOR, Insecure Direct Object Reference, is the older term commonly used when a direct object identifier is exposed and the authorization check is missing or broken.

Owner: Alice

Resource: /orders/101

- 1) Alice -> GET /orders/101 -> expected allowed
- 2) Bob -> GET /orders/101 -> expected denied

If Bob receives Alice-specific data and verification confirms it, the finding becomes high-confidence BOLA.

Key interview point

BOLA is not “there is an integer in the URL.” The vulnerability is the missing authorization decision around the object that identifier points to.

5.2 BFLA

BFLA means Broken Function Level Authorization. Here the problem is not which object is accessed; it is whether a low-privileged identity can invoke a function reserved for a higher-privileged role.

Required role: admin

Attacker role: user

user -> DELETE /admin/users/42

If the server performs the admin-only action, that is a BFLA candidate.

5.3 Privilege escalation / mass assignment

The scanner looks for security-sensitive writable fields such as role, roles, admin, is_admin, permissions, or account type. But a successful PATCH is not enough to prove the escalation.

- 1) user -> PATCH /users/me {"role":"admin"}
- 2) user -> GET /admin-only-endpoint
- 3) If the new capability is observable -> confirmed escalation

5.4 Parameter tampering

The tool tests sensitive client-controlled fields such as price, amount, balance, discount, or status. The security question is whether the server blindly trusts a value that should be calculated or validated server-side.

```
{  
  "product": "laptop",  
  "price": 0.01  
}
```

5.5 Business logic / workflow abuse

Business logic requires knowledge of intended sequencing. The project therefore uses configuration-driven workflows and tests concrete patterns such as skipping a required step or replaying a sensitive step.

Expected: Create cart -> Payment -> Checkout

Test: Create cart -> Checkout

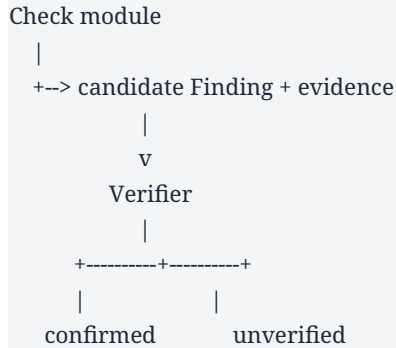
Expected: Redeem coupon once

Test: Redeem coupon -> Redeem same coupon again

6. Verification and evidence

The part that turns a scanner into a testing system

The project deliberately separates candidate generation from verification. This makes every check simple and keeps false-positive reduction in one auditable component.



1. **REPRODUCE:** issue the suspicious request again to rule out a one-off response or race condition.
2. **INSPECT CONTENT:** do not trust status codes alone. An empty body or an “access denied” message behind a 200 should not count as a leak.
3. **CHECK IDENTITY MARKERS:** for BOLA, look for configured strings that identify the victim principal’s data.
4. **RE-TEST CAPABILITIES:** for privilege escalation, invoke a restricted endpoint after tampering to observe whether the user actually gained privileges.
5. **KEEP UNCERTAIN RESULTS VISIBLE:** candidates that do not clear the verification bar stay in the report as unverified instead of being silently discarded.

Why this matters

A security report is more useful when a reviewer can reproduce the finding from concrete request/response evidence instead of trusting a one-line scanner verdict.

Evidence handling

Every HTTP call is captured as an evidence record containing method, URL, status, timing, request/response bodies, and relevant headers. Sensitive authentication material such as Authorization, Cookie, and X-API-Key is redacted before the evidence is stored or rendered.

7. Attack-path correlation

Turning separate findings into impact

A vulnerability list tells me what is wrong. An attack path tells me how the problems can compound. The project builds paths only from confirmed findings, using a small set of grounded relationships rather than guessing arbitrary chains.

LOW-PRIV USER

|

| privilege escalation

v

ADMIN-LIKE CAPABILITY

|

| broken function-level authorization

v

ADMIN-ONLY ENDPOINT

Another example is a reconnaissance chain: a BOLA finding exposes data that reveals an identifier, then another confirmed authorization flaw uses that information to reach a more sensitive capability.

Design constraint

Attack paths are additive, not a replacement for individual findings. If no grounded chain exists, every confirmed finding is still reported on its own.

8. The vulnerable lab and Docker

How I made the project demonstrable

A security tool is much easier to evaluate when it ships with a target that is intentionally vulnerable. The repository includes a Flask lab with seeded vulnerabilities and safe control endpoints. That gives the scanner something deterministic to test without requiring a third-party system.

Component	Purpose
Flask lab	Acts as the deliberately vulnerable target.
OpenAPI lab spec	Describes the target API surface to the scanner.
Lab roles configuration	Defines the seeded personas, ownership, and authorization expectations.
Safe control endpoints	Provide negative cases so the test suite can catch false positives.
Docker Compose	Packages the lab and scanner so the demo is reproducible and isolated.

The README says the included lab contains eight seeded vulnerabilities and two safe control endpoints. The integration suite checks both that expected vulnerabilities are confirmed and that safe controls are not flagged.

```
./scripts/run_demo.sh

# or manually
cd lab && python app.py &
python3 scripts/generate_lab_config.py http://localhost:8000
apiattack scan --spec examples/openapi_lab.yaml \
  --config examples/roles_lab.yaml \
  --out ./report --yes-i-am-authorized
```

Safety boundary

The authorization flag is a guardrail, not legal permission. The project is intended only for systems you own or are explicitly authorized to test, and the deliberately vulnerable lab should remain isolated.

9. Testing strategy

How I tried to prevent both missed bugs and fake bugs

The test strategy is stronger than “the scanner runs without crashing.” The included lab lets the suite check expected vulnerabilities and safe behavior at the same time.

Test objective	Example assertion
Functional correctness	A seeded BOLA is detected and confirmed.
False-positive control	A safe endpoint is not flagged as BOLA/BFLA.
Verification correctness	A 200 response containing an error-shaped message is not treated as a confirmed leak.
Privilege verification	A role-field mutation only becomes confirmed when a restricted capability changes.
Integration stability	End-to-end scan succeeds against the in-process lab.

```
pytest -q
pytest --cov=apiattack -q
```

The architecture also documents a development lesson: early BOLA testing could incorrectly treat an admin reading another user’s data as a vulnerability. The implementation now excludes the highest-privilege role(s) from acting as the attacker persona in BOLA probes. This is a good example of using observed test behavior to refine the scanner’s threat model.

10. A complete demo walk-through

What happens when I run the scanner

6. Clone the repository and install the Python package into a virtual environment.
7. Start the intentionally vulnerable Flask lab locally or through Docker.
8. Generate or prepare the role configuration so the scanner knows the identities, ownership, privilege ranks, and workflow expectations.
9. Inspect the OpenAPI specification to confirm the endpoints that will be tested.
10. Run the scan with an explicit authorization acknowledgement.
11. The parser builds Endpoint objects and the config loader builds ScanConfig.
12. The runner invokes the selected check modules. Each module performs actual HTTP requests under the relevant role.
13. Candidates are collected with request/response evidence.
14. The verifier replays and inspects candidates. Some become confirmed; others stay unverified.
15. Confirmed findings are sent to the attack-path builder.
16. The report writers generate Markdown, HTML, and JSON outputs.

```
apiattack inspect-spec --spec examples/openapi_lab.yaml
```

```
apiattack scan \  
--spec examples/openapi_lab.yaml \  
--config examples/roles_lab.yaml \  
--out ./report \  
--yes-i-am-authorized
```

What I would show an interviewer

I would not only show a final “8 vulnerabilities found” screen. I would open one finding and walk through the exact request, role, response, verification step, and evidence that caused the result to be confirmed.

11. Design decisions and trade-offs

Decision	Why	Trade-off
OpenAPI-driven	Creates a structured, reviewable attack surface.	Requires a usable API spec; undocumented/shadow endpoints are outside the normal entry path.
Config-driven roles	Makes ownership and intended permissions explicit.	Someone must author accurate role/ownership data.
Deterministic checks	Reproducible and auditable.	Less adaptive than an autonomous agent.
Separate verification layer	Centralizes false-positive reduction.	More HTTP traffic and more implementation complexity.
Evidence-first reporting	Makes findings independently reviewable.	Reports need careful secret redaction.
Grounded attack-path heuristics	Produces readable impact chains without inventing facts.	Cannot infer every possible multi-step attack.
Dockerized vulnerable lab	Fast, repeatable demonstrations.	Adds container/runtime complexity.

Strongest design choice

The strongest architectural decision is the candidate -> verification split. It gives the checks a simple contract while making “confirmed” mean more than “HTTP 200 returned.”

12. Limitations and roadmap

What the project does not solve yet

A strong interview presentation should include limitations. Claiming that a small deterministic scanner solves the complete API security problem would weaken the project, not strengthen it.

Current limitation	Reason / future direction
No generic SQLi/XSS/fuzzing engine	The scope is authorization testing; broader vulnerability discovery is a separate problem.
Business logic needs configuration	Business intent cannot generally be reconstructed safely from an OpenAPI schema.
No automated OAuth2/OIDC login flow	The current model expects usable role credentials in configuration.
GraphQL not covered	The input model is OpenAPI/HTTP endpoint oriented.
No SARIF output yet	Could be added for CI/security-gate integration.
No autonomous LLM control loop	Determinism and auditability are intentional design goals.

The repository’s roadmap names GraphQL support, OAuth2/OIDC automation, additional rate-limit/business-logic checks, and SARIF as possible future work.

13. Interview presentation script

A polished 60-90 second explanation

Use this as a script, then personalize it

I built an API security testing tool focused on authorization vulnerabilities. The input is an OpenAPI specification plus a YAML configuration containing different test identities, their credentials, the resources they own, their privilege levels, and the endpoints or workflows they should be allowed to use.

The scanner parses the API into a normalized endpoint inventory and runs separate checks for BOLA/IDOR, broken function-level authorization, privilege escalation through mass assignment, sensitive parameter tampering, and configurable business-logic abuse. I deliberately separate detection from verification: a check creates a candidate when behavior looks suspicious, then a verification layer replays the request, inspects response content, checks identity markers, and for privilege escalation re-tests a restricted capability.

Confirmed findings are correlated into grounded attack paths so I can explain not only what is broken but how multiple authorization flaws can compound. I also built an intentionally vulnerable Flask lab and Docker workflow so the whole system can be reproduced locally, and the tool produces Markdown, HTML, and JSON evidence-based reports.

A 30-second version

Short version

“I built a Python-based, OpenAPI-driven API authorization tester. It uses multiple role identities to test BOLA, BFLA, privilege escalation, parameter tampering, and workflow abuse, then verifies suspicious responses and correlates confirmed findings into attack paths. I also built a vulnerable Flask lab and Dockerized demo so the scanner can be tested end-to-end.”

14. Questions I must be able to answer

Likely interviewer questions with technically defensible answers

What is BOLA?

Broken Object Level Authorization: the server authenticates the caller but fails to verify that the caller is authorized for the specific object identified by the request.

What is BFLA?

Broken Function Level Authorization: a caller with insufficient privilege can invoke an operation reserved for a higher-privileged role.

Why not rely on HTTP status codes?

Because 200 can still carry an error, empty body, redirect page, or soft denial. Response semantics need to be verified.

Why multiple identities?

Authorization is relational: the same endpoint can be valid for one role and invalid for another.

How do you know a BOLA leak is real?

Reproduce the request and look for configured victim identity markers or other strong content evidence.

How do you prove privilege escalation?

Change a security-sensitive property, then directly test a restricted capability and observe whether access changed.

Why is business logic configuration-driven?

Because OpenAPI can describe an operation but usually cannot express the intended business sequence of operations.

Why separate checks from verification?

To keep each detector simple and to centralize false-positive reduction in one auditable stage.

Why exclude admins from BOLA attacker probes?

Because an admin may legitimately access another user's objects. Testing with admin credentials can manufacture a false positive.

What does Docker contribute?

Reproducibility and isolation for the scanner plus the intentionally vulnerable lab.

What would you improve next?

OAuth2/OIDC automation, GraphQL support, more business-logic checks, SARIF/CI output, and broader test coverage.

Is this an autonomous pentesting agent?

No. The architecture is deterministic, fixed-strategy, and reviewable by design.

15. Glossary

Terms to know before presenting the project

Term	Meaning in this project
API	Application Programming Interface: the HTTP surface the scanner tests.
OpenAPI	A machine-readable description of API paths, methods, parameters, and schemas.
Authentication	Proves who the caller is.
Authorization	Decides what the authenticated caller is allowed to do.
BOLA	Broken Object Level Authorization.
IDOR	Insecure Direct Object Reference; a common way BOLA manifests.
BFLA	Broken Function Level Authorization.
Mass assignment	Binding attacker-controlled fields directly into privileged server-side properties.
Parameter tampering	Manipulating client-supplied values that should be validated or derived by the server.
Business logic flaw	Abuse of application rules or workflow sequencing rather than a purely technical parsing bug.
Finding	A scanner result describing suspicious or confirmed security behavior.
Candidate	A finding emitted by a check that has not yet passed verification.
Confirmed	A finding that passed the project's verification criteria.
Evidence	The redacted request/response record supporting a finding.
Attack path	A grounded chain showing how multiple confirmed findings can compound.

Appendix A - Repository evidence map

The following repository files are the best places to study before an interview:

- README.md - project goals, quickstart, feature list, CLI examples, testing, roadmap, and scope.
- docs/ARCHITECTURE.md - the pipeline, design principles, verification model, false-positive lesson, attack-path logic, and extension points.
- src/apiattack/spec_parser.py - OpenAPI normalization into Endpoint objects.
- src/apiattack/config/ - role/ownership/workflow configuration loading.

- src/apiattack/engine/ - HTTP client and scan orchestration.
- src/apiattack/checks/ - the five vulnerability-testing strategies.
- src/apiattack/verification/ - candidate confirmation logic.
- src/apiattack/attack_path/ - confirmed-finding correlation.
- src/apiattack/reporting/ - report generation.
- lab/ - deliberately vulnerable Flask API.
- tests/ - unit and integration coverage.
- pyproject.toml - package metadata, dependencies, and CLI entry point.

Appendix B - Final interview checklist

- Can I explain authentication vs authorization without mixing them up?
- Can I walk through a BOLA example request by request?
- Can I explain why BFLA is different from BOLA?
- Can I explain why a 200 response is not proof of authorization bypass?
- Can I explain how the verifier reduces false positives?
- Can I explain how privilege escalation is capability-tested?
- Can I explain why business logic is configuration-driven?
- Can I explain the OpenAPI -> roles -> checks -> verification -> attack path -> report pipeline?
- Can I explain what Docker is doing in the demo?
- Can I state the project's limitations without overselling it?
- Can I open the repository and point to the implementation of each major stage?

Sources used to build this documentary

Primary source: the public repository supplied by the user:

<https://github.com/d3zathon/claude-AATi-test>

Repository files reviewed include README.md, docs/ARCHITECTURE.md, pyproject.toml, and the project tree under src/apiattack/, lab/, examples/, and tests/. The documentary intentionally distinguishes repository behavior from claims about personal authorship.